

SOM-Seq: A Python Toolbox for Single-Cell Sequencing Simulation and Self-Organizing Map Analysis

Zachary Caterer ^{1,2,3*}, Madeline Pernat ^{4*}, and Victoria Hurd ^{5*}

¹ Department of Chemical and Biological Engineering, University of Colorado Boulder, Boulder, CO, USA ² Department of Biomedical Informatics, University of Colorado Anschutz Medical Campus, Aurora, CO, USA ³ Biofrontiers Institute Interdisciplinary Biology PhD Program, University of Colorado Boulder, Boulder, CO, USA ⁴ Department of Civil, Environmental, and Architectural Engineering, University of Colorado Boulder, Boulder, CO, USA ⁵ Department of Aerospace Engineering Sciences, University of Colorado Boulder, Boulder, CO, USA ¶ Corresponding author * These authors contributed equally.

DOI: [10.xxxxxx/draft](https://doi.org/10.xxxxxx/draft)

Software

- [Review](#) 
- [Repository](#) 
- [Archive](#) 

Editor: [Mojtaba Barzegari](#) 

Reviewers:

- [@Ruslan-Kurmashev](#)
- [@riccabolla](#)
- [@AmaanArif25](#)

Submitted: 16 July 2026

Published: unpublished

License

Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License ([CC BY 4.0](#)).

Summary

SOM-Seq is an open-source Python toolbox that integrates two complementary workflows for single-cell genomics research: synthetic sequencing data generation (Seq_Sim) and Self-Organizing Map (SOM) based clustering and visualization (SOM). The Seq_Sim module generates realistic pseudo-bulk single-cell datasets with configurable cell-type compositions, batch effects, disease states, and differential expression patterns, adapted from simulation approaches developed in the Zhang Lab ([Inamo & Young, 2024](#)). The SOM module provides a high-level Python class built on MiniSom ([Vettigli, 2018](#)) that handles data scaling, automated hyperparameter tuning, topographic quality metrics, and publication-quality visualizations including component planes and categorical overlays. Both modules expose command-line interfaces (CLIs), making them composable within broader bioinformatics pipelines or usable as standalone tools.

Statement of Need

Self-Organizing Maps (SOMs), introduced by Kohonen ([Kohonen, 1990](#)), produce a discrete two-dimensional grid of neurons in which neighboring neurons represent similar regions of the input feature space, explicitly encoding topological structure between clusters. Despite this interpretive advantage, SOMs remain underutilized in the single-cell community. A key barrier is the absence of a well-tested, end-to-end Python package that combines SOM fitting with single-cell-style simulation, reducing the friction required to benchmark SOM-based clustering against other methods on controlled synthetic data.

SOM-Seq addresses this gap in two ways. First, Seq_Sim generates synthetic datasets that statistically mirror real single-cell data—including heterogeneous cell-type proportions, batch variability, and disease-associated fold changes—providing researchers with a reproducible, ground-truth-labeled environment for method comparison without requiring access to patient data. Second, the SOM module wraps the complete SOM workflow (scaling, training, metric evaluation, and visualization) into a clean Python API and CLI, lowering the expertise required to apply SOM-based analysis to tabular omics data.

Together, these modules enable researchers to simulate a dataset with known structure, fit a SOM, and immediately evaluate clustering quality using Percent Variance Explained (PVE) and topographic error—a capability not available in existing single-cell analysis frameworks.

State of the Field

High-dimensional single-cell RNA sequencing (scRNA-seq) data requires dimensionality reduction and clustering to reveal biologically meaningful structure (Luecken & Theis, 2019). Established tools such as Seurat (Hao et al., 2021) and Scanpy (Wolf et al., 2018) are the standard for single-cell analysis, typically pairing graph-based community detection with t-SNE (Maaten & Hinton, 2008) or UMAP (McInnes et al., 2018) for visualization. While powerful, these methods embed data into a continuous low-dimensional space that does not explicitly preserve the topological distances between clusters, making it difficult to reason about the relative proximity (i.e., similarity) of cell populations.

At the algorithmic level, general-purpose SOM libraries such as MiniSom (Vettigli, 2018) provide no single-cell simulation capability, while single-cell simulators such as the SCORPIO framework from the Zhang Lab (Inamo & Young, 2024), on which Seq_Sim is based, are not paired with an SOM analysis workflow. To our knowledge, no existing package combines SOM-based clustering with single-cell-style data simulation in one tool, so researchers wishing to benchmark SOM-based clustering must currently assemble a simulation tool and a separate SOM library themselves. SOM-Seq fills this gap by integrating both capabilities behind a single, consistent API and CLI.

Software Design

SOM-Seq's design reflects two deliberate trade-offs. First, rather than requiring real patient-derived single-cell data, Seq_Sim generates statistically realistic synthetic data with known ground-truth cell-type labels; this sacrifices biological realism for reproducibility and removes the privacy and data-access barriers that complicate benchmarking clustering methods on human subject data. Second, rather than reimplementing SOM training, the SOM module wraps the existing, well-tested MiniSom library (Vettigli, 2018) and adds the scaling, tuning, metric, and visualization steps that a complete single-cell workflow requires; this keeps the core training algorithm maintained upstream while concentrating SOM-Seq's engineering effort on the parts of the workflow specific to omics data analysis.

Sequence Simulation (Seq_Sim)

The Seq_Sim module generates synthetic single-cell datasets by constructing a subject-level metadata table (age, sex, disease status, batch) and a cell-type composition matrix. Cell counts for major and rare cell populations are drawn from uniform distributions parameterized by user-supplied standard deviations and relative abundances. Disease-associated differential abundance is introduced by adding or removing cells of specified types in proportion to a configurable fold-change parameter. Pseudo-feature expression matrices are then generated per cell by combining cluster-specific signal, disease variance, and individual-level variance, with additive Gaussian noise controlled by a cluster ratio parameter. All random operations accept a seed argument to guarantee reproducibility.

Key configurable parameters exposed via config.yml or CLI include:

- num_samples: number of subjects to simulate
- fold_change: magnitude of disease-associated differential abundance
- n_major_cell_types, n_minor_cell_types: cell-type composition
- n_features: number of pseudo-expression features per cell
- n_batches, prop_disease, prop_sex: study design parameters

Self-Organizing Maps (SOM)

The SOM module wraps MiniSom (Vettigli, 2018) into a SOM Python class that manages the full analysis workflow:

- 87 1. **Input validation:** type and range checks on all constructor arguments.
- 88 2. **Scaling:** z-score or min-max normalization with corresponding inverse transforms stored
- 89 for weight unscaling.
- 90 3. **Training:** configurable grid dimensions, rectangular or hexagonal topology, and Gaussian
- 91 or bubble neighborhood functions.
- 92 4. **Metrics:** PVE measures how much of the input variance is captured by the neuron
- 93 weight vectors; topographic error ([Kohonen, 1990](#)) quantifies how often a data point's
- 94 two best-matching units are non-adjacent on the grid.
- 95 5. **Hyperparameter tuning:** when multiple values are supplied for any hyperparameter, the
- 96 CLI evaluates all combinations and selects the configuration maximizing $PVE - 100 \times$
- 97 $topographic_error$.
- 98 6. **Visualization:** component plane plots (one heatmap per input feature) and categorical
- 99 overlay plots (one heatmap per categorical variable), saved as publication-quality PNG
- 100 files.

101 Usage

102 Generating Sequencing Data

```
python Seq_Sim/seq_sim.py \
  --num_samples 30 \
  --fold_change 0.5 \
  --config_file Seq_Sim/config.yml
```

103 Fitting a SOM

```
python SOM/som.py \
  -t data/sim_data_pseudo_feature_num_samples_30_fc_0.5.csv \
  -c data/sim_data_latent_data_num_samples_30_fc_0.5.csv \
  -o output/ \
  -s zscore \
  -x 5 -y 4 \
  -p hexagonal \
  -n gaussian \
  -e 100
```

104 Python API

```
import pandas as pd
from SOM.utils.som_utils import SOM

train = pd.read_csv("data/sim_data_pseudo_feature_num_samples_30_fc_0.5.csv")
meta = pd.read_csv("data/sim_data_latent_data_num_samples_30_fc_0.5.csv")

som = SOM(
    train_dat=train,
    other_dat=meta,
    scale_method="zscore",
    x_dim=5,
    y_dim=4,
    topology="hexagonal",
    neighborhood_fnc="gaussian",
    epochs=100,
)
som.train_map()
```

```
print(f"PVE: {som.calculate_percent_variance_explained():.1f}%")
print(f"Topographic error: {som.calculate_topographic_error():.3f}")
som.plot_component_planes(output_dir="output/")
som.plot_categorical_data(output_dir="output/")
```

Testing and Documentation

SOM-Seq ships with a pytest test suite covering both modules, including input validation, scaling round-trips, training, metric calculations, and plot generation. Continuous integration via GitHub Actions runs the full suite on each push. API documentation is hosted on Read the Docs.

Research Impact Statement

Beyond its origin as a course project, SOM-Seq is in active use: the SOM module is currently being applied to real single-cell data as part of ongoing biomedical informatics research at the University of Colorado Anschutz Medical Campus, where it is being evaluated as a topology-preserving alternative to graph-based clustering pipelines. Community-readiness is further supported by a pytest test suite exercising both modules, continuous integration on every push, published API documentation, an OSI-approved open-source license, and packaging for installation via PyPI, lowering the barrier for other groups to adopt or extend the software.

Acknowledgements

This work originated as part of the University of Colorado Boulder course **CSCI 6118: Software Engineering for Scientists**, created by **Dr. Ryan Layer** and taught by **Dr. Erik Johnson**. We thank both Dr. Layer and Dr. Johnson for their guidance and for creating the collaborative environment that made this project possible. This work was also supported in part by the U.S. National Science Foundation under **Award No. 2022138**.

AI usage disclosure

We used Anthropic's Claude (Sonnet 4.6) to assist with refining documentation, including adding clarity and removing redundant information. We also used Anthropic's Claude (Sonnet 5, via Claude Code) during pre-submission preparation to identify and fix two command-line interface bugs (an argument-parsing crash in the SOM CLI and an invalid function call in the Seq_Sim CLI), to add continuous integration workflows for package build validation and PyPI publishing, and to correct inconsistencies in the LICENSE, and README. All AI-assisted content and code changes were reviewed by the authors, who validated correctness and made all core design decisions.

References

- Hao, Y., Hao, S., Andersen-Nissen, E., Mauck, W. M., Zheng, S., Butler, A., Lee, M. J., Wilk, A. J., Darby, C., Zager, M., Hadley, P., Steger, M., Nair, P., Bhardwaj, N., Buenrostro, J. D., Shekhar, K., Bhatt, D. L., Barnard, C., & Satija, R. (2021). Integrated analysis of multimodal single-cell data. *Cell*, 184(13), 3573–3587.e29. <https://doi.org/10.1016/j.cell.2021.04.048>
- Inamo, J., & Young, J. (2024). *SCORPIO: Simulated single-cell omics with realistic phenotypes and integrated outcomes*. <https://github.com/fanzhanglab/pyCellPhenoX>

- 141 Kohonen, T. (1990). The self-organizing map. *Proceedings of the IEEE*, 78(9), 1464–1480.
142 <https://doi.org/10.1109/5.58325>
- 143 Luecken, M. D., & Theis, F. J. (2019). Current best practices in single-cell RNA-seq analysis:
144 A tutorial. *Molecular Systems Biology*, 15(6), e8746. [https://doi.org/10.15252/msb.](https://doi.org/10.15252/msb.20188746)
145 [20188746](https://doi.org/10.15252/msb.20188746)
- 146 Maaten, L. van der, & Hinton, G. (2008). Visualizing data using t-SNE. *Journal of Machine*
147 *Learning Research*, 9, 2579–2605. <https://jmlr.org/papers/v9/vandermaaten08a.html>
- 148 McInnes, L., Healy, J., & Melville, J. (2018). UMAP: Uniform manifold approximation and
149 projection for dimension reduction. *arXiv Preprint arXiv:1802.03426*. [https://doi.org/10.](https://doi.org/10.48550/arXiv.1802.03426)
150 [48550/arXiv.1802.03426](https://doi.org/10.48550/arXiv.1802.03426)
- 151 Vettigli, G. (2018). *MiniSom: Minimalistic and NumPy-based implementation of the self*
152 *organizing map*. <https://github.com/JustGlowing/minisom>
- 153 Wolf, F. A., Angerer, P., & Theis, F. J. (2018). SCANPY: Large-scale single-cell gene expression
154 data analysis. *Genome Biology*, 19(1), 15. <https://doi.org/10.1186/s13059-017-1382-0>

DRAFT